Step 3: .env.example

Story S0.1.1 Monorepo with Docker Compose (GitHub issue #2) | DocQA | Oct 16, 2026

Goal

Create `.env.example`, the committed template that documents **every** setting DocQA uses, and a local `.env` copy with your real values. The S0.1.1 acceptance criterion is: `.env.example` *documents every variable.*

File	Committed to Git?	Purpose
`.env.example`	Yes	Template with safe placeholder values and a comment for each variable
`.env`	No (already in .gitignore)	Your real values. Docker Compose reads it in step 5

Where: both files live in the repo root, `D:\DocReaderQA`. `.env.example` already exists and is empty. No Python code reads these variables yet; that starts with the typed config in S0.2.1. In this step you only write the file.

Step 3.1 - Fill in .env.example

Open `.env.example` in VS Code and paste the content below. Variables are grouped by the service that uses them. Later stories already need some of them, so the file stays the single reference.

```
# ============================================================
# DocQA environment variables
# Copy this file to .env and adjust the values:
#   Copy-Item .env.example .env
# Never commit .env. Never put real secrets in this file.
# ============================================================

# ---------- App ----------
# development | production (turns dev-only endpoints on/off)
APP_ENV=development

# ---------- PostgreSQL (read by the postgres container) ----------
# User, password and database created on the first start
POSTGRES_USER=docqa
POSTGRES_PASSWORD=docqa
POSTGRES_DB=docqa

# ---------- Backend: database ----------
# Async SQLAlchemy URL. Must use the same user/password/db as above.
# Host "postgres" is the compose service name. Outside Docker use localhost.
DATABASE_URL=postgresql+asyncpg://docqa:docqa@postgres:5432/docqa

# ---------- Backend: Redis job queue ----------
# Host "redis" is the compose service name. Outside Docker use localhost.
REDIS_URL=redis://redis:6379/0

# ---------- Ollama (runs on the Windows host, decision D10) ----------
OLLAMA_BASE_URL=http://host.docker.internal:11434
LLM_MODEL=qwen3:8b
EMBEDDING_MODEL=nomic-embed-text
# Must match the embedding model; fixes the VECTOR(n) column (D5)
EMBEDDING_DIM=768

# ---------- Auth ----------
# Generate one: python -c "import secrets; print(secrets.token_urlsafe(48))"
```

```
JWT_SECRET=change-me
ACCESS_TOKEN_MINUTES=30
REFRESH_TOKEN_DAYS=7

# ---------- Uploads and storage ----------
# Folder inside the api/worker containers (mounted from ./storage)
STORAGE_DIR=/data/storage
MAX_UPLOAD_MB=50
MAX_PAGES=500

# ---------- CORS ----------
# Comma-separated origins allowed to call the API
CORS_ORIGINS=http://localhost:5173

# ---------- Frontend (Vite) ----------
# Only variables starting with VITE_ are visible to the browser code
VITE_API_BASE_URL=http://localhost:8000/api/v1
```

Step 3.2 - Understand each variable

Variable	Used by	What it means
`APP_ENV`	api	`development` or `production`. Dev-only endpoints (S1.5.4) check it
`POSTGRES_USER / _PASSWORD / _DB`	postgres	The official Postgres image creates this user and database on the very first start
`DATABASE_URL`	api, worker	How the backend connects. `+asyncpg` selects the async driver. Must repeat the same user, password and database name
`REDIS_URL`	api, worker	Where the ARQ job queue lives. Your worker already reads it
`OLLAMA_BASE_URL`	api, worker	Ollama runs on Windows, not in Docker. `host.docker.internal` is how a container reaches the host
`LLM_MODEL`	api	Chat model for answers and query rewriting (decision D8)
`EMBEDDING_MODEL / _DIM`	api, worker	Embedding model and its vector size; 768 for nomic-embed-text (D5)
`JWT_SECRET`	api	Signs login tokens. Anyone with it can forge logins, so use a long random value
`ACCESS_TOKEN_MINUTES / REFRESH_TOKEN_DAYS`	api	Token lifetimes (S0.3.2, S0.3.4)
`STORAGE_DIR`	api, worker	Where uploaded PDFs are saved inside the containers
`MAX_UPLOAD_MB / MAX_PAGES`	api, worker	Upload size limit (API) and page limit (worker, D12)
`CORS_ORIGINS`	api	Lets the browser app on port 5173 call the API on port 8000
`VITE_API_BASE_URL`	frontend	API address used by the React app. Vite only exposes variables that start with `VITE_`

Step 3.3 - Create your local .env

In the VS Code terminal, from the repo root:

```
cd D:\DocReaderQA
Copy-Item .env.example .env
```

Then replace `JWT_SECRET=change-me` in `.env` with a real random value. Generate one with the backend venv active:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Paste the output after `JWT_SECRET=`. Everything else can keep the example values for local development.

Step 3.4 - Verify

Check that Git ignores .env but sees .env.example:

```
git check-ignore -v .env
git status --short
```

- `git check-ignore` should print a line pointing at `.gitignore`, which means `.env` is ignored.
- `git status` should list `.env.example` but **not** `.env`.

A full check with `docker compose config` comes in step 5, once `docker-compose.yml` exists.

Rules for this file

- **No real secrets in `.env.example`.** It is committed and your repo is public.
- **One `KEY=value` per line**, no spaces around `=`, no quotes needed.
- **Comments go on their own line** starting with #. Text after a value on the same line can become part of the value.
- **Keep both files in sync.** Every new setting goes into `.env.example` (with a comment) and into your `.env`.
- **Database credentials appear twice** (`POSTGRES_*` and `DATABASE_URL`). Change them together.

Running the backend outside Docker

The service names `postgres` and `redis` only resolve inside the compose network. When you run `uvicorn` or `arq` directly on Windows (as in step 2), use `localhost` instead:

```
DATABASE_URL=postgresql+asyncpg://docqa:docqa@localhost:5432/docqa
REDIS_URL=redis://localhost:6379/0
OLLAMA_BASE_URL=http://localhost:11434
```

> Your step 2 worker already falls back to `redis://localhost:6379/0` when `REDIS_URL` is not set, so no change is needed yet.

Done when

- [ ] `.env.example` contains every variable above, each with a comment
- [ ] `.env` exists in the repo root with a real `JWT_SECRET`
- [ ] `git status` shows `.env.example` and does not show `.env`

Suggested commit

```
git add .env.example
```

```
git commit -m "Add .env.example documenting all settings (refs #2)"
```

Do not run `git add .` without checking `git status` first, so `.env` can never slip in.

Common problems

Symptom	Cause	Fix
`.env` shows in `git status`	`.gitignore` missing the entry or file saved with another name	Check `.gitignore` has `.env`; check the file isn't `.env.txt`
Windows saves it as `.env.txt`	Created through Explorer with extensions hidden	Create it with `Copy-Item` in the terminal or VS Code's New File
Postgres ignores a changed password	Credentials are applied only on the first start	Reset the database volume: `docker compose down -v` (deletes data)
Browser shows CORS error later	Frontend URL not in `CORS_ORIGINS`	Add the exact origin, e.g. `http://localhost:5173`

Next: step 4, the frontend placeholder (Vite + React + TypeScript in `docqa-frontend`).